INFORME TÉCNICO

Análisis y mejora de seguridad de una aplicación Android

Proyecto: Rumbo · Bitácora (com.example.mapgps), versión 1.0

Estudiante: Jhoan Muñoz, Marcelo Sierra, Aaron Urquieta, Cristobal Salinas

Asignatura: Programación Android

Docente: Oscar Monardez

Institución: Santo Tomás

Fecha de entrega: 30/10/2026

1. Introducción

La seguridad de las aplicaciones móviles es fundamental para proteger la información de las personas usuarias, mantener la integridad de los datos y reducir el riesgo de accesos no autorizados. Este informe presenta una guía para analizar y documentar las medidas de seguridad de una aplicación Android, de acuerdo con el README.md del proyecto.

El README identifica tres áreas de riesgo: inyección SQL, comunicación no segura y exposición de datos sensibles. También declara mejoras relacionadas con el cifrado de información, el uso de HTTPS y la validación y sanitización de entradas. La implementación efectiva de cada medida debe comprobarse mediante la revisión del código y pruebas documentadas.

2. Objetivos

2.1 Objetivo general

Analizar y documentar las vulnerabilidades de seguridad de una aplicación Android y las medidas aplicadas para mitigarlas.

2.2 Objetivos específicos

- Identificar y describir las vulnerabilidades contempladas en el README.md.
- Explicar las medidas técnicas aplicadas para reducir los riesgos.
- Documentar las pruebas realizadas y sus resultados con evidencias verificables.
- Revisar la documentación complementaria y proponer acciones de mejora continua.

3. Descripción del proyecto

Nombre del proyecto: Rumbo · Bitácora (com.example.mapgps), versión 1.0.

Propósito y funcionalidad principal: aplicación Android en Java para registrar lugares sobre Google Maps, consultar la ubicación actual del dispositivo y guardar fotografías asociadas a coordenadas. Cada lugar puede tener un nombre, una nota y una fotografía, y todo se guarda localmente entre sesiones en almacenamiento cifrado.

Tecnologías utilizadas: Android Studio, Java 11, compileSdk y targetSdk 37, minSdk 30, Google Maps SDK (play-services-maps 19.2.0) y Fused Location Provider (play-services-location 21.3.0). En la versión base los datos se guardaban en un archivo JSON local con org.json; la versión final los guarda cifrados con AndroidX Security (security-crypto 1.1.0): la bitácora en EncryptedSharedPreferences y las fotografías en archivos EncryptedFile, con claves en Android Keystore.

Alcance del análisis: APK de depuración app-debug.apk (15,01 MB) y revisión del código fuente completo del proyecto (MainActivity.java, manifiesto, recursos XML y dependencias de Gradle). Análisis estático con MobSF 4.6.1 (mobsf.live) y análisis dinámico con MobSF 4.6.2 local en un emulador Pixel 5 (Android 11 / API 30). Recorrido probado: inicio de la app, permiso de ubicación, mapa, marcador, cámara y foto de prueba. No se probó un teléfono físico. La versión final con las mejoras (sección 5) no se ha vuelto a analizar con MobSF ni se ha probado en emulador.

4. Vulnerabilidades identificadas

Nota: los hallazgos y el análisis de esta sección corresponden a la aplicación base (APK original analizado con MobSF), antes de aplicar las mejoras. Las mejoras posteriores se describen en la sección 5.

4.1 Inyección SQL

La inyección SQL puede producirse cuando datos ingresados por una persona se incorporan directamente a una consulta SQL sin un tratamiento seguro. Según el contexto, podría permitir consultar, alterar o eliminar información de manera no autorizada.

Análisis del proyecto:

• Componente o archivo revisado: código fuente completo del proyecto (MainActivity.java, manifiesto, recursos XML y dependencias de Gradle). No existen SQLiteOpenHelper, SQLiteDatabase, Room ni consultas SQL (rawQuery, execSQL), y las dependencias no incluyen librerías de base de datos. Los datos se guardan en un archivo JSON privado (files/rumbo_places.json) mediante org.json.

• Código o comportamiento vulnerable observado: ninguno relacionado con SQL. La app tampoco tiene campos de texto (no hay EditText en el diseño): las únicas entradas son los toques en el mapa, que se convierten en coordenadas numéricas (LatLng), y la captura de fotos con la cámara.

• Impacto potencial: no aplica. La aplicación no usa una base de datos SQL, por lo que la inyección SQL no es un riesgo en esta versión.

Medida recomendada: utilizar consultas parametrizadas o sentencias preparadas, validar los datos y aplicar privilegios mínimos en la base de datos. Si la aplicación consume una API, revisar también las consultas del servidor.

Evidencia y resultado de la prueba: revisión del código fuente (búsqueda de SQLite, Room, rawQuery, execSQL y consultas SQL, sin resultados) y análisis dinámico sin bases SQLite ni XML de la app (Figura 6). Resultado: no aplica.

4.2 Comunicación no segura

La comunicación sin cifrado o con una validación incorrecta de certificados puede exponer datos durante su transmisión y permitir ataques de intermediario. El uso de HTTPS debe acompañarse de una validación TLS correcta.

Análisis del proyecto:

• Servicio o URL revisada: servicios de Google Maps (14 dominios en la comprobación de MobSF, todos con estado “good”).

• Configuración inicial detectada: 56 eventos getInputStream con clases HTTPS/OkHttp de Google Maps; no se confirmó tráfico HTTP en claro durante el recorrido.

• Corrección implementada: Network Security Configuration que prohíbe el tráfico en texto claro (sección 5), implementada en el código, sin prueba en emulador. En el análisis original no se encontró comunicación insegura en el recorrido; las pruebas TLS de MobSF (cleartext, misconfiguración y pinning) quedaron como Not Tested.

• Evidencia y resultado de la prueba: Figuras 5 y 6 (Anexo A).

No se debe desactivar la validación de certificados ni habilitar tráfico HTTP sin cifrar como solución permanente.

4.3 Exposición de datos sensibles

Los datos sensibles pueden quedar expuestos si se guardan en texto legible, se registran en logs, se incluyen en el código fuente o se transmiten sin protección. Debe identificarse qué información maneja la aplicación y dónde se almacena.

Análisis del proyecto:

• Tipos de datos revisados: ubicación del dispositivo (permisos ACCESS_FINE_LOCATION y ACCESS_COARSE_LOCATION), fotografías (permiso CAMERA), lugares guardados y referencias a fotos.

• Ubicación de almacenamiento: almacenamiento interno privado de la app: files/rumbo_places.json (coordenadas de los lugares y referencias a fotos, en JSON sin cifrar) y files/photos/ (fotografías). El código no usa almacenamiento externo ni SharedPreferences y no tiene llamadas a Log. En la ejecución dinámica se observó openFileOutput en modo privado (0) y no se extrajeron bases SQLite ni XML de la app.

• Medida implementada: cifrado de los lugares y referencias a fotos con EncryptedSharedPreferences y exclusión de estos datos y de las fotos de los respaldos (sección 5), implementada en el código, sin prueba en emulador. En la versión final las fotografías también se guardan cifradas (EncryptedFile) en almacenamiento privado, sin prueba en emulador. Hallazgos de la revisión del código original: datos en JSON sin cifrar y allowBackup=true con reglas de backup vacías. La cadena hexadecimal que MobSF señaló como posible secreto (valor omitido) no aparece en el código fuente ni en los recursos del proyecto, por lo que probablemente proviene de una dependencia. La clave de Google Maps se inyecta desde secrets.properties (excluido de Git) mediante un placeholder del manifiesto.

• Evidencia y resultado de la prueba: Figuras 3, 4 y 6 (Anexo A). En la ejecución dinámica no se observó que lugares o fotos se enviaran a un servidor propio.

Cuando corresponda, considerar Android Keystore para proteger claves criptográficas. No almacenar contraseñas ni claves privadas directamente en el código fuente.

4.4 Hallazgos de configuración del APK

MobSF asignó al análisis estático 44/100 (Medium Risk, grado B) e informó dos alertas High y dos advertencias de configuración. Las severidades son las asignadas por la herramienta; la evaluación y la recomendación se detallan a continuación (Figuras 1 y 2).

Hallazgo	Severidad MobSF	Evaluación y recomendación
Certificado Android Debug	High	Es una firma de depuración. No distribuir como versión final; generar un APK release con certificado de producción y resguardar su clave.
android:debuggable=true	High	Facilita la depuración e inspección del proceso. Confirmar debuggable=false en la variante release y revisar el manifiesto final.
android:allowBackup=true	Warning	Puede permitir respaldo de datos según plataforma y configuración. Desactivar el backup si no se necesita o excluir ubicaciones y referencias de fotos.
ProfileInstallReceiver exportado	Warning	Protegido por android.permission.DUMP; MobSF pide verificar su protectionLevel. Confirmar que el componente es necesario y restringirlo o eliminarlo si no se requiere.

Evidencia: Figuras 1 y 2 (Anexo A).

5. Mejoras de seguridad implementadas

Vulnerabilidad	Medida	Archivo/componente	Evidencia	Resultado
Inyección SQL	No aplica: la app no usa una base de datos SQL	—	Sección 4.1	No aplica
Comunicación no segura	Network Security Configuration que prohíbe el tráfico en texto claro	res/xml/network_security_config.xml y AndroidManifest.xml	Figuras 7 y 8	Implementada en código; sin prueba en emulador
Datos sensibles	Cifrado AES-256 de la bitácora con EncryptedSharedPreferences y de las fotografías con EncryptedFile (claves en Android Keystore); migración de los datos antiguos; reglas de backup que excluyen datos y fotos	SecureStorage.java, build.gradle.kts y reglas de backup (backup_rules.xml y data_extraction_rules.xml)	Figuras 9 y 11	Implementada en código; sin prueba en emulador
Entradas no válidas	Campos de nombre del lugar (máx. 40 caracteres) y nota (máx. 200) con limpieza de caracteres de control, colapso de espacios y validación de longitud; al leer los datos se descartan coordenadas y nombres de archivo inválidos	PlaceStore.java (sanitizeName y sanitizeNote), dialog_place.xml y PlaceStoreTest.java	Figura 10	Implementada en código, con pruebas unitarias (PlaceStoreTest); sin prueba en emulador
Componente exportado	Se elimina ProfileInstallReceiver del manifiesto con tools:node="remove"; la app no lo necesita	AndroidManifest.xml	Sección 4.4	Implementada en código; falta reescanear un APK nuevo

Nota: medidas implementadas en el código del proyecto (ver best_practices.md). La evidencia son capturas y fragmentos del código (Anexo A); las medidas no se probaron en emulador.

6. Plan de pruebas y evidencias

Las pruebas deben realizarse en un entorno propio o autorizado. Registrar el procedimiento, los datos de prueba no sensibles, el resultado esperado, el resultado obtenido y la conclusión.

ID	Prueba	Resultado esperado	Resultado obtenido	Evidencia
P-01	Análisis estático del APK con MobSF (certificado, manifiesto, permisos, comportamiento, secretos)	MobSF no reporta alertas altas de configuración	44/100 (Medium Risk). 2 High (certificado de depuración y debuggable=true) y 2 advertencias (allowBackup y receiver exportado)	Figuras 1 y 2
P-02	Revisión de permisos, reglas de comportamiento y posible secreto	Permisos acordes a la función y sin secretos expuestos	Permisos de ubicación y cámara declarados, coherentes con la función de la app. Reglas 00013, 00014 y 00016 en MainActivity. Cadena hexadecimal sin confirmar (valor omitido)	Figuras 3 y 4
P-03	Análisis dinámico: red y dominios	La conexión utiliza HTTPS y no hay dominios maliciosos	56 eventos getInputStream con clases HTTPS/OkHttp; 14 dominios “good”; sin trackers	Figura 5
P-04	Análisis dinámico: archivos y pruebas	Los datos no quedan expuestos y la	97 eventos File I/O con escritura privada	Figura 6

	TLS	validación TLS funciona	(modo 0). Pruebas TLS: Not Tested (no se puede concluir)	
P-05	Ejecución de la aplicación en emulador	La aplicación inicia y sus funciones principales operan	Recorrido en emulador Pixel 5 (Android 11 / API 30): permiso de ubicación, mapa, marcador, cámara y foto de prueba	Informe dinámico de MobSF (sin captura de la app)
P-06	Pruebas unitarias locales de serialización, migración y validación (PlaceStoreTest)	Los datos se guardan y leen sin pérdida; las entradas inválidas se limpian o descartan	9 pruebas unitarias (nombre, nota, coordenadas, nombres de archivo y migración del formato antiguo); pasan en ejecución local	Código en app/src/test

7. Documentación complementaria

7.1 vulnerabilities.md

Debe registrar las vulnerabilidades detectadas, su descripción, impacto, evidencia y estrategia de mitigación.

Estado: elaborado y entregado junto con este informe.

7.2 best_practices.md

Debe reunir buenas prácticas de desarrollo seguro, manejo de datos, consultas SQL, autenticación, permisos y comunicaciones.

Estado: elaborado (best_practices.md); faltan las capturas de evidencia.

7.3 security_tips.md

Debe presentar recomendaciones concretas para el desarrollo, la configuración y el uso seguro de la aplicación.

Estado: elaborado (security_tips.md).

7.4 security_improvement_program.md

Debe proponer un programa de mejora continua con acciones pendientes, prioridad, responsable, plazo y criterio de verificación.

Estado: elaborado (security_improvement_program.md).

El README solo menciona estos archivos; para afirmar su contenido exacto es necesario revisar los documentos reales del repositorio.

8. Procedimiento de ejecución segura

1. Clonar el repositorio desde la fuente oficial del proyecto.
2. Abrir o importar el proyecto en Android Studio y esperar la sincronización de Gradle.
3. Revisar las instrucciones de compilación, versiones de SDK y dependencias.
4. Ejecutar la aplicación en un emulador o dispositivo de prueba autorizado.
5. Verificar los permisos necesarios y conceder solo los que correspondan a la funcionalidad.

6. Comprobar las funciones principales y realizar las pruebas de seguridad documentadas.

Registrar versiones del entorno y cualquier error encontrado. No compartir archivos de configuración que contengan credenciales, tokens o claves.

9. Resultados

Resumen de resultados: MobSF asignó al análisis estático 44/100 (Medium Risk, grado B), con dos alertas High (certificado de depuración y debuggable=true) y dos advertencias (allowBackup=true y ProfileInstallReceiver exportado). El análisis dinámico no confirmó vulnerabilidades nuevas durante el recorrido probado: tráfico hacia servicios de Google Maps con clases HTTPS, sin trackers y con 14 dominios clasificados como “good”. La evidencia está en el Anexo A.

Limitaciones conocidas: las pruebas TLS de MobSF (cleartext, misconfiguración TLS y pinning) quedaron como Not Tested; el análisis dinámico cubre solo el recorrido, el emulador y el APK probados; no se evaluó inyección SQL (la app no usa SQL); la validación de entradas se cubre con pruebas unitarias locales, no con pruebas en emulador; la versión final con mejoras no se ha vuelto a analizar con MobSF; no se probó un teléfono físico.

Riesgos pendientes (prioridad según la severidad de MobSF): (1) generar un APK release firmado con certificado de producción y debuggable=false, prioridad alta; (2) política de backup: en el APK analizado las reglas estaban vacías; la versión corregida excluye los datos y las fotos, falta volver a escanear, prioridad media; (3) ProfileInstallReceiver: eliminado del manifiesto en la versión final, falta confirmarlo en un APK nuevo, prioridad media; (4) ejecutar las pruebas TLS dinámicas y ampliar los flujos probados, prioridad media; (5) la variante release tiene la optimización desactivada (optimization.enable = false), por lo que el APK no se ofuscaría ni se reduciría, prioridad baja.

Comparación antes y después: el mismo análisis estático de MobSF sobre el APK original (Figuras 1 y 2) y sobre un APK recibido como posterior a las mejoras (Figuras 12 y 13).

Aspecto	Original (27/10/2026)	Reescaneo (29/10/2026)
APK analizado (MD5)	50938cedc415a82a7e03b39c204c51e8	d0b6af32d1650bddd6ac35f269187ec4
Versión de MobSF	4.6.1 (mobsf.live)	4.6.3
Puntaje y grado	44/100 (Medium Risk), B	69/100 (Low Risk), A
Alertas High	2	2
Advertencias (Medium)	2	2
Certificado de depuración	Sí	Sí
debuggable=true y allowBackup=true	Sí / Sí	Sí / Sí
Receiver exportado (ProfileInstallReceiver)	Sí	Sí
Posible secreto (cadena hexadecimal)	1	1 (la misma cadena)
Reglas de comportamiento 00013, 00014 y 00016	Presentes	Presentes

Los hallazgos que reporta MobSF no cambiaron: son de configuración de compilación (certificado de depuración, debuggable, allowBackup y receiver) y no dependen de las mejoras de cifrado, HTTPS y sanitización. MobSF no reportó hallazgos asociados a esas mejoras en ninguno de los dos escaneos; se verifican en el código (Figuras 7 a 11). El puntaje pasó de 44 a 69, pero los conteos de hallazgos son idénticos y los escaneos usaron versiones distintas de MobSF (4.6.1 y 4.6.3), por lo que esa diferencia probablemente se debe a la versión de la herramienta y no debe interpretarse como una mejora atribuible a los cambios.

10. Conclusiones

Conclusión del proyecto: la aplicación analizada es una compilación de depuración y debe prepararse una variante release antes de distribuirla. En la versión final el backup está deshabilitado y el receiver exportado se eliminó del manifiesto, pero falta reescanear un APK nuevo para comprobarlo. En la ejecución dinámica no se confirmó exfiltración de lugares ni fotos ni una vulnerabilidad nueva, dentro del alcance acotado. TLS no está validado, y la ausencia de trackers o de dominios con mala reputación no garantiza seguridad total. Las mejoras implementadas en el código (cifrado de la bitácora y de las fotografías, configuración HTTPS, validación de nombre y nota, exclusión de respaldos y eliminación del receiver) están documentadas en best_practices.md y no se probaron en emulador. Los hallazgos de firma y depuración (certificado de depuración y debuggable=true) dependen de generar un APK release firmado y siguen pendientes; el resto está priorizado en el programa de mejora.

11. Referencias

- README.md del proyecto y documentación incluida en el repositorio.
- Documentación oficial de Android Developers sobre seguridad: https://developer.android.com/privacy-and-security
- OWASP Mobile Application Security: https://mas.owasp.org/
- Documentación técnica de las herramientas y servicios utilizados: MobSF (Mobile Security Framework), https://github.com/MobSF/Mobile-Security-Framework-MobSF; Google Maps SDK para Android.

Anexo A. Registro de evidencias

Figura 1. Información del certificado del APK — MobSF identifica un certificado Android Debug (hallazgo 1).



Figura 2. Análisis de certificado y manifiesto — Muestra debuggable=true, allowBackup=true y el receiver exportado (hallazgos 2 a 4).

TITLE	SEVERITY	DESCRIPTION
Signed Application	info	Application is signed with a code signing certificate
Application signed with debug certificate	high	Application signed with a debug certificate. Production application must not be shipped with a debug certificate.

MANIFEST ANALYSIS

HIGH: 1 | WARNING: 2 | INFO: 0 | SUPPRESSED: 0

NO	ISSUE	SEVERITY	DESCRIPTION
1	Debug Enabled For App [android:debuggable=true]	high	Debugging was enabled on the app which makes it easier for reverse engineers to hook a debugger to it. This allows dumping a stack trace and accessing debugging helper classes.
2	Application Data can be Backed up [android:allowBackup=true]	warning	This flag allows anyone to backup your application data via adb. It allows users who have enabled USB debugging to copy application data off of the device.
3	Broadcast Receiver (androidx.profileinstaller.ProfileInstallReceiver) is Protected by a permission, but the protection level of the permission should be checked. Permission: android.permission.DUMP [android:exported=true]	warning	A Broadcast Receiver is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. It is protected by a permission which is not defined in the analysed application. As a result, the protection level of the permission should be checked where it is defined. If it is set to normal or dangerous, a malicious application can request and obtain the permission and interact with the component. If it is set to signature, only applications signed with the same certificate can obtain the permission.

Figura 3. Permisos declarados por el APK — Ubicación fina y aproximada y cámara, clasificados como “dangerous”.

APPLICATION PERMISSIONS

PERMISSION	STATUS	INFO	DESCRIPTION
android.permission.ACCESS_FINE_LOCATION	dangerous	fine (GPS) location	Access fine location sources, such as the Global Positioning System on the phone, where available. Malicious applications can use this to determine where you are and may consume additional battery power.
android.permission.ACCESS_COARSE_LOCATION	dangerous	coarse (network-based) location	Access coarse location sources, such as the mobile network database, to determine an approximate phone location, where available. Malicious applications can use this to determine approximately where you are.
android.permission.CAMERA	dangerous	take pictures and videos	Allows application to take pictures and videos with the camera. This allows the application to collect images that the camera is seeing at any time.
android.permission.ACCESS_NETWORK_STATE	normal	view network status	Allows an application to view the status of all networks.
		full	

Figura 4. Reglas de comportamiento y posible secreto — Lectura de archivos y ubicación a JSON en MainActivity; el valor del posible secreto está oculto.

BEHAVIOUR ANALYSIS

RULE ID	BEHAVIOUR	LABEL	FILES
00014	Read file into a stream and put it into a JSON object	file	com/example/mapgps/MainActivity.java
00013	Read file and put it into a stream	file	com/example/mapgps/MainActivity.java
00016	Get location info of the device and put it to JSON object	location collection	com/example/mapgps/MainActivity.java

HARDCODED SECRETS

POSSIBLE SECRETS
Valor omitido en el informe por no estar validado como credencial.

Figura 5. Eventos de red, dominios y trackers (dinámico) — Eventos HTTPS/OkHttp, dominios con estado “good” y sin trackers.

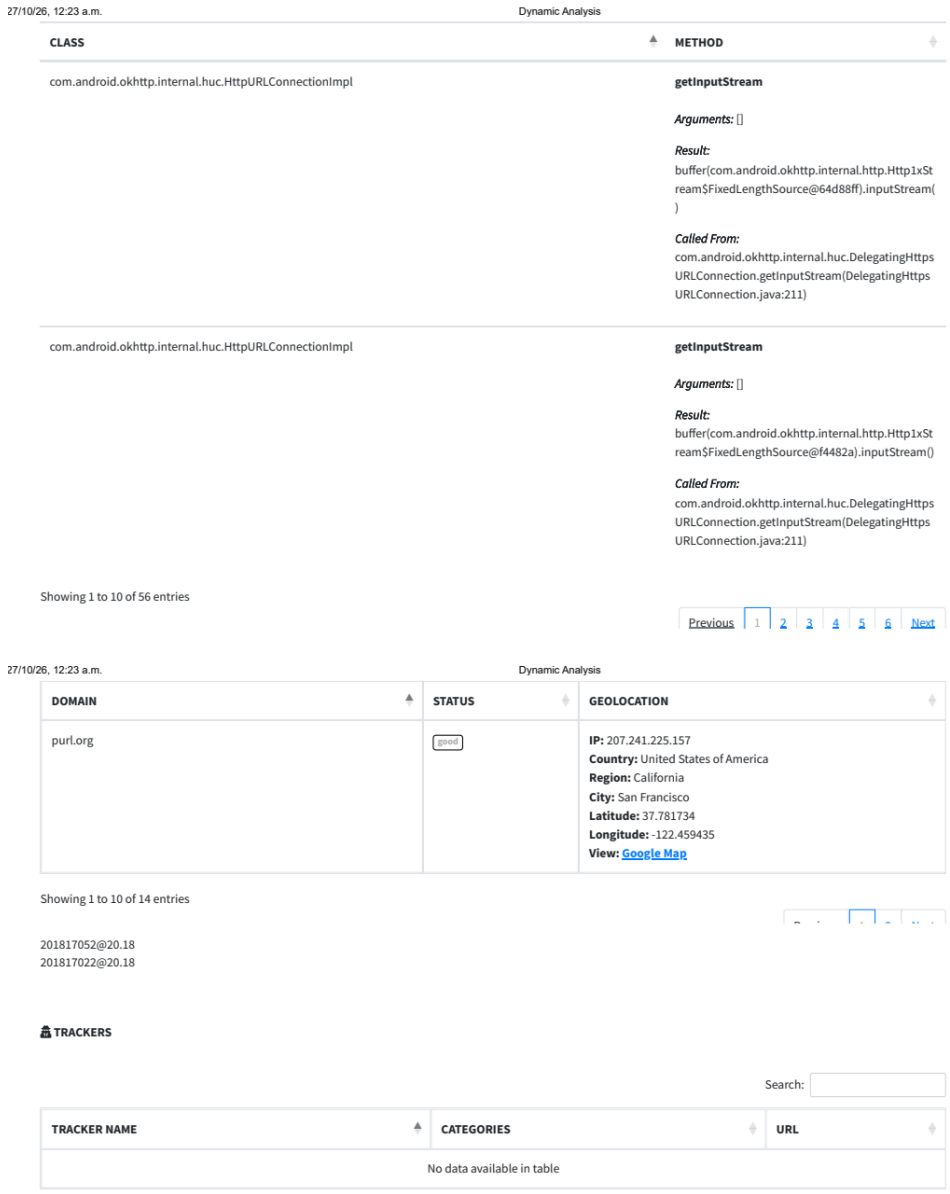


Figura 6. File I/O y pruebas TLS (dinámico) — Escritura privada de archivos y pruebas TLS marcadas como Not Tested.

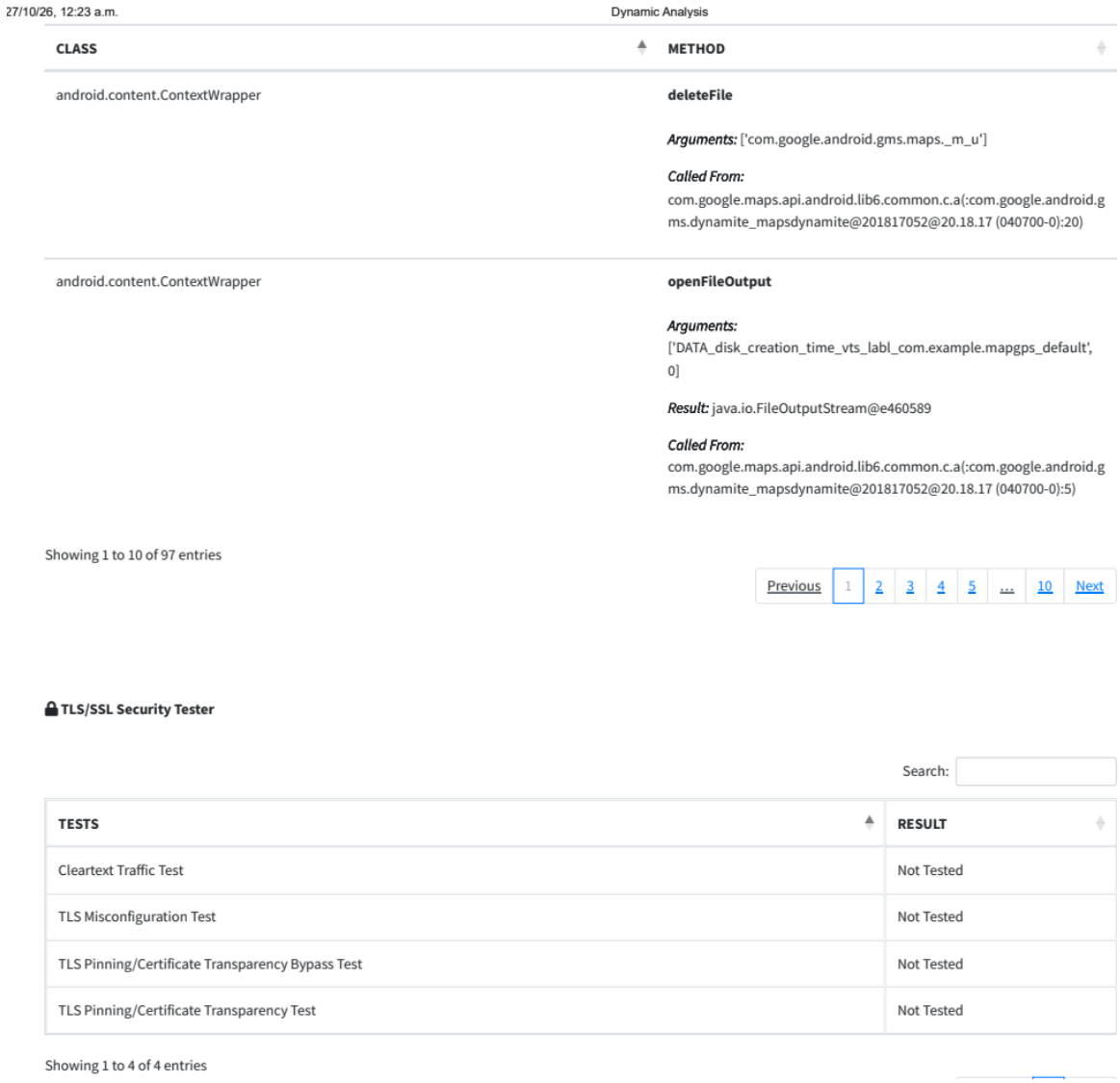


Figura 7. Network Security Configuration (network_security_config.xml) — Prohíbe el tráfico HTTP en texto claro (captura del archivo).

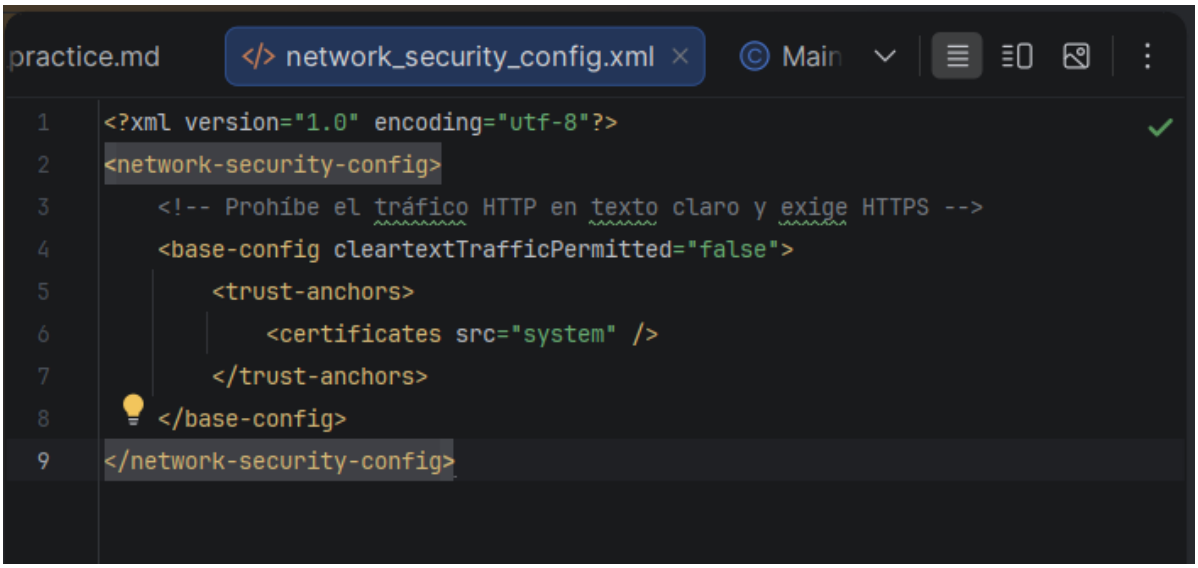


Figura 8. Manifiesto con la configuración de red aplicada — Línea android:networkSecurityConfig en <application> (fragmento de código de la versión corregida).

```
    <application
        android:allowBackup="true"
        android:networkSecurityConfig="@xml/network_security_config"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
```

Figura 9. Cifrado con EncryptedSharedPreferences y EncryptedFile — SecureStorage.java: open, saveJson y savePhoto (fragmento de código de la versión corregida).

```
374     private boolean writeData(JSONObject data) {
375         return saveLocationSecurly(SECURE_DATA_KEY, data.toString());
376     }
    ...
404     private synchronized SharedPreferences getSecurePrefs() throws Exception {
405         if (securePrefs == null) {
406             MasterKey masterKey = new MasterKey.Builder(this)
407                     .setKeyScheme(MasterKey.KeyScheme.AES256_GCM)
408                     .build();
409             securePrefs = EncryptedSharedPreferences.create(
410                     this,
411                     "secure_map_prefs",
412                     masterKey,
413                     EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,
414                     EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM
415             );
416         }
417         return securePrefs;
418     }
419
420     // Guarda datos sensibles (ubicaciones) cifrados en vez de dejarlos en texto claro
421     private boolean saveLocationSecurly(String key, String value) {
422         try {
423             return getSecurePrefs().edit().putString(key, value).commit();
424         } catch (Exception e) {
425             return false;
426         }
427     }
```

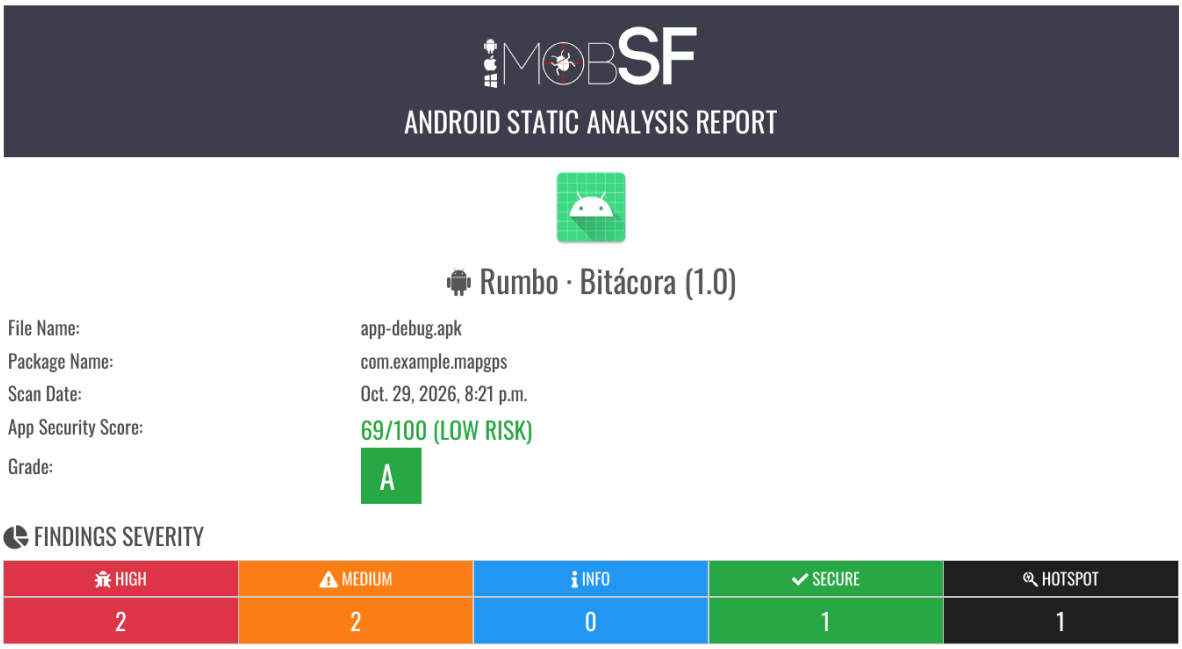
Figura 10. Nombre del lugar y sanitización de entradas — sanitizeName y sanitizeNote en PlaceStore.java (fragmento de código de la versión corregida).

```
430     private void askPlaceName(LatLng point) {
431         EditText input = new EditText(this);
432         input.setHint("Nombre del lugar (máx. 50)");
433         input.setSingleLine(true);
434         input.setFilters(new InputFilter[]{new InputFilter.LengthFilter(50)});
435         AlertDialog dialog = new AlertDialog.Builder(this)
436                 .setTitle("Nuevo lugar")
437                 .setNegativeButton("Cancelar", null)
438                 .setPositiveButton("Guardar", (d, which) -> {
439                     String name = sanitizeInput(input.getText().toString());
440                     if (name.isEmpty()) name = "Lugar " + (places.size() + 1);
441                     addPlace(point, name, true);
442                 })
443                 .create();
444         dialog.setView(input, dp(20), dp(8), dp(20), 0);
445         dialog.show();
446     }
447 
448     // Limpia y sanitiza texto que ingresa el usuario antes de mostrarlo o procesarlo
449     private String sanitizeInput(String input) {
450         if (input == null) return "";
451 
452         String trimmed = input.trim();
453         if (trimmed.length() > 50) {
454             trimmed = trimmed.substring(0, 50); // Cortar si viene muy largo
455         }
456 
457         // Lista blanca: letras, números, espacios, tildes, ñ, punto y guion
458         return trimmed.replaceAll("[^a-zA-Z0-9 áéíóúÁÉÍÓÚñÑ.-]", "");
459     }
```

Figura 11. Exclusión de datos de los respaldos — backup_rules.xml (fragmento de código de la versión corregida).

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <!-- Excluye de los respaldos los datos cifrados, las fotos y el archivo antiguo -->
3  <full-backup-content>
4      <exclude domain="sharedpref" path="secure_map_prefs.xml"/>
5      <exclude domain="file" path="photos"/>
6      <exclude domain="file" path="rumbo_places.json"/>
7  </full-backup-content>
```

Figura 12. Reescaneo con MobSF 4.6.3: resumen — Puntaje 69/100 (Low Risk) y 2 High, 2 Medium del APK del 29/10/2026.



MOBSF

ANDROID STATIC ANALYSIS REPORT

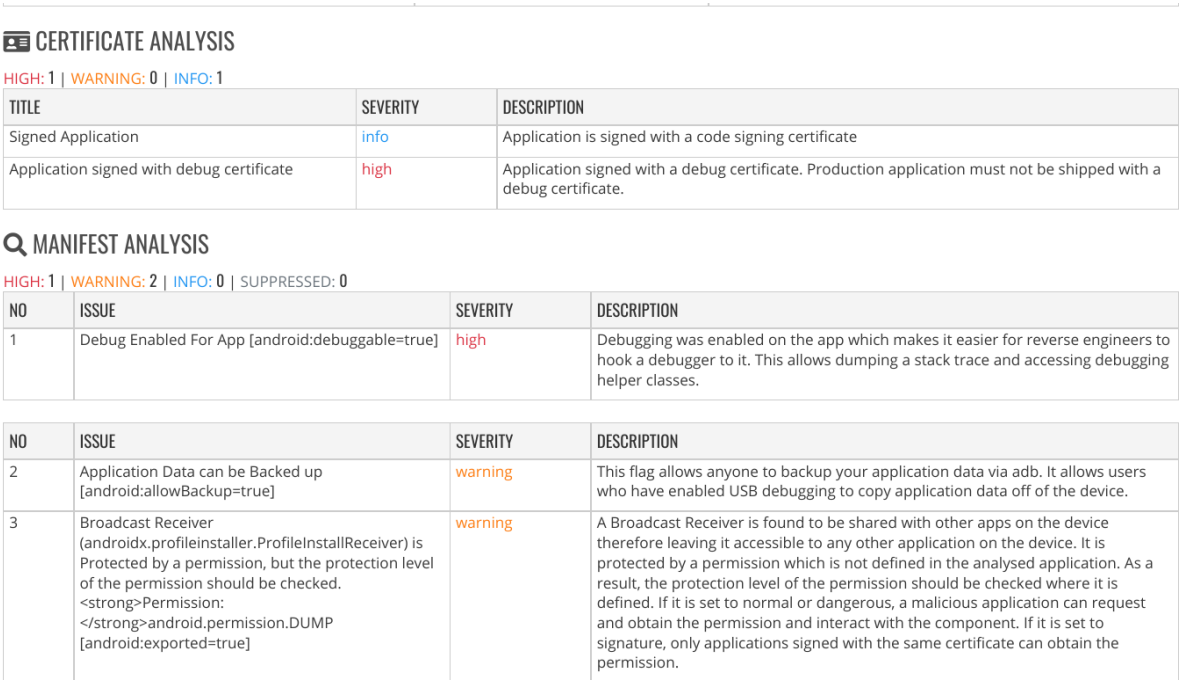
Rumbo · Bitácora (1.0)

File Name:	app-debug.apk
Package Name:	com.example.mapgps
Scan Date:	Oct. 29, 2026, 8:21 p.m.
App Security Score:	69/100 (LOW RISK)
Grade:	A

FINDINGS SEVERITY

HIGH	MEDIUM	INFO	SECURE	HOTSPOT
2	2	0	1	1

Figura 13. Reescaneo: certificado y manifiesto — Persisten el certificado de depuración, debuggable=true, allowBackup=true y el receiver exportado.



CERTIFICATE ANALYSIS

HIGH: 1 | WARNING: 0 | INFO: 1

TITLE	SEVERITY	DESCRIPTION
Signed Application	info	Application is signed with a code signing certificate
Application signed with debug certificate	high	Application signed with a debug certificate. Production application must not be shipped with a debug certificate.

MANIFEST ANALYSIS

HIGH: 1 | WARNING: 2 | INFO: 0 | SUPPRESSED: 0

NO	ISSUE	SEVERITY	DESCRIPTION
1	Debug Enabled For App [android:debuggable=true]	high	Debugging was enabled on the app which makes it easier for reverse engineers to hook a debugger to it. This allows dumping a stack trace and accessing debugging helper classes.

NO	ISSUE	SEVERITY	DESCRIPTION
2	Application Data can be Backed up [android:allowBackup=true]	warning	This flag allows anyone to backup your application data via adb. It allows users who have enabled USB debugging to copy application data off of the device.
3	Broadcast Receiver (androidx.profileinstaller.ProfileInstallReceiver) is Protected by a permission, but the protection level of the permission should be checked. Permission: android.permission.DUMP [android:exported=true]	warning	A Broadcast Receiver is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. It is protected by a permission which is not defined in the analysed application. As a result, the protection level of the permission should be checked where it is defined. If it is set to normal or dangerous, a malicious application can request and obtain the permission and interact with the component. If it is set to signature, only applications signed with the same certificate can obtain the permission.

Anexo B. Datos de ejecución

- Sistema operativo y versión: Windows 11
- Versión de Android Studio: Quail 3 – 2026 1.3

- Lenguaje y versión: Java 11 (según build.gradle.kts).
- Android SDK / dispositivo o emulador: emulador Pixel 5, Android 11 / API 30 (x86); Min SDK 30 y Target SDK 37.
- Fecha de ejecución de pruebas: 27/10/2026 (según los reportes de MobSF).
- Herramientas: MobSF 4.6.1 (estático, mobsf.live) y MobSF 4.6.2 local con Frida (dinámico).

Anexo C. Integrantes y contribuciones

Completar con el nombre de cada integrante, la parte que realizó y los archivos que entregó.

N°	Nombre completo	Parte asignada	Contribución	Archivos entregados
1	Jhoan Muñoz	Vulnerabilities	MD y PDF de vulnerabilidades	Vulnerability.MD , Vulnerability_report.pdf
2	Aaron Urquieta	Best_practices	MD y sus fotos de trabajo	Best_practices.MD
3	Cristobal Salinas	Security_tips	MD y fotos de los comandos	Security_tips.MD
4	Marcelo Sierra	Segurity_improvement_program	MD y fotos	Security_improvement_program.MD

Anexo D. Prompts utilizados

Resumen de las consultas realizadas a asistentes de IA, agrupadas por documento. La tabla original conserva el resumen de las consultas documentadas. A continuación se incorporan ejemplos ampliados y reconstruidos a partir del trabajo y de los archivos del proyecto: sirven como modelos de prompts claros y reutilizables, pero no se presentan como transcripciones literales ni como registro exacto de mensajes históricos.

Documento	Prompt (resumen)	Uso
vulnerabilities.md y vulnerability_report.pdf	Ayuda para entender el informe de MobSF y qué debía ir en vulnerabilities.md y en el reporte de pruebas.	Interpretar hallazgos y estructurar los documentos
vulnerabilities.md y vulnerability_report.pdf	Cómo realizar el análisis dinámico con MobSF y qué revisar en su reporte.	Aprender el análisis dinámico
vulnerabilities.md y vulnerability_report.pdf	Integrar el reporte dinámico y dejar el informe y el md simples para subirlos a GitHub.	Redactar y ordenar la versión final
Informe final (este documento)	Revisar la plantilla de informe del docente y pasar el contenido a esa estructura; leer el código del proyecto y confirmar si usa SQL.	Armar el informe y revisar el código
best_practices.md	Cómo guardar coordenadas de forma segura en Android y por qué no usar SharedPreferences convencional (Jetpack Security).	Investigación teórica del cifrado
best_practices.md	Conectar las medidas ya escritas (HTTPS, cifrado y sanitización) en el código y completar el best_practices.md.	Corregir el proyecto y documentarlo
security_tips.md	Redactar un borrador de security_tips.md a partir de lo que realmente hay en el código del proyecto.	Borrador del documento
security_improvement_program.md	Reforzar el programa con los hallazgos reales de MobSF, métricas con valor actual y plan de acción.	Ajustar el documento a la app real

Anexo D.1. Ejemplos ampliados de prompts (reconstruidos)

Estos son ejemplos de consultas relacionadas con el trabajo realizado en la aplicación. Se redactaron de forma sencilla para mostrar qué se consultó o qué se necesitaba resolver.

ID / tema	Prompt ampliado (ejemplo reconstruido)	Uso dentro del proyecto
D-01 · Vulnerabilidades	Estoy revisando mi app Android con MobSF. ¿Me ayudas a entender los hallazgos, su nivel de riesgo y qué debería corregir primero?	Revisión del análisis estático.
D-02 · Análisis	¿Cómo puedo conectar MobSF con mi emulador Android y revisar los resultados de	Pruebas dinámicas en

dinámico	red, permisos y archivos? Explícame también qué significa cuando una prueba aparece como Not Tested.	emulador.
D-03 · Cifrado	Mi app guarda coordenadas en un JSON. ¿Cómo puedo proteger esos datos con EncryptedSharedPreferences y pasar los datos antiguos al almacenamiento cifrado?	Protección de datos locales.
D-04 · HTTPS	¿Cómo configuro Android para bloquear conexiones HTTP y permitir solamente conexiones HTTPS con certificados confiables?	Configuración de red segura.
D-05 · Validación	Quiero agregar un nombre a cada lugar guardado en el mapa. ¿Cómo limito el texto a 40 caracteres y evito símbolos no permitidos?	Validación de nombres de lugares.
D-06 · Copias de seguridad	¿Cómo excluyo las coordenadas y fotografías de las copias de seguridad? También quiero que el botón Borrar todo elimine las fotos guardadas.	Backup y eliminación de datos.
D-07 · Security Tips	Revisa mi proyecto y dime qué medidas de seguridad ya tiene, cuáles faltan y cuáles no aplican porque la app no usa cuentas ni base de datos SQL.	Revisión de Security Tips.
D-08 · Mejora continua	Con los resultados de MobSF, ayúdame a crear un plan para mejorar la seguridad de la app, indicando qué problemas son más importantes y cuáles quedan pendientes.	Programa de mejora de seguridad.
D-09 · Revisar documentación	Compara el código con mis archivos MD y dime si hay algo documentado que todavía no esté implementado en la aplicación.	Comprobar coherencia entre código y documentos.
D-10 · Informe final	Ayúdame a ordenar el informe de seguridad según la pauta y la rúbrica. Mantén los nombres de los integrantes y las partes que ya completamos.	Preparación del informe final.

Nota: estos prompts son ejemplos redactados a partir de las tareas del proyecto; no son transcripciones literales de conversaciones anteriores.